

Методичка по созданию Unit test на C# (Практическая работа №4)

Оглавление

<i>Теория.....</i>	<i>2</i>
<i>Объяснение на практике: Тестирование простого калькулятора.....</i>	<i>3</i>
<i>Справка (Пример работы)</i>	<i>7</i>
<i>Практическое задание</i>	<i>9</i>

Теория

Юнит-тест (модульный тест) — это автоматический скрипт, который проверяет работоспособность самой маленькой изолированной части кода, например отдельной функции или метода.

Зачем нужны юнит-тесты?

1. Быстрое обнаружение ошибок: Ошибки находятся на самых ранних этапах разработки, еще до того, как код попадет к пользователю.
2. Безопасный рефакторинг: Можно менять и улучшать внутреннее устройство кода, не боясь сломать уже работающие функции.
3. Понятность кода: Тесты служат живой документацией, показывая, как именно должен работать конкретный метод.

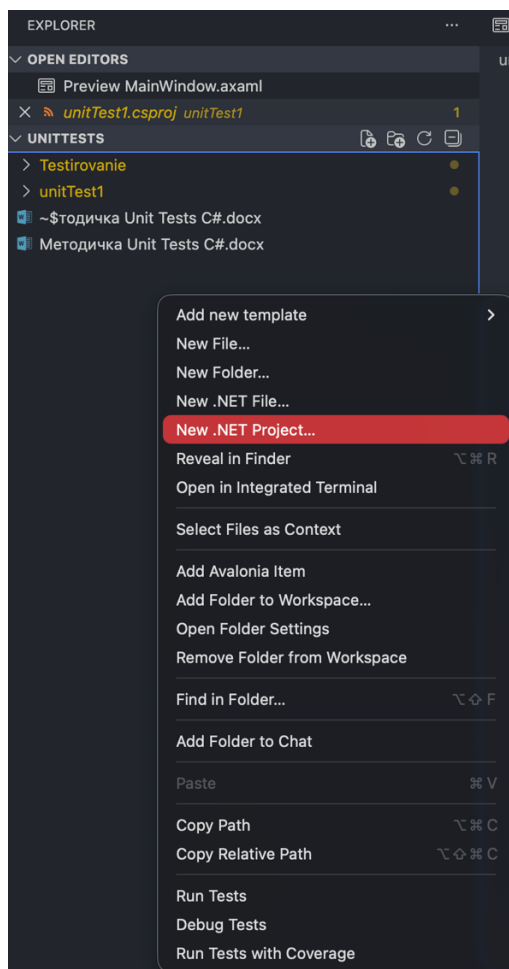
Особенности и свойства

1. Изоляция: Тестируемый кусок кода полностью отделен от базы данных, файлов и других внешних сервисов с помощью специальных заглушек (моков).
2. Высокая скорость: Проверка сотен таких тестов занимает считанные секунды.
3. Авторство: Обычно такие тесты пишут сами разработчики программного обеспечения.

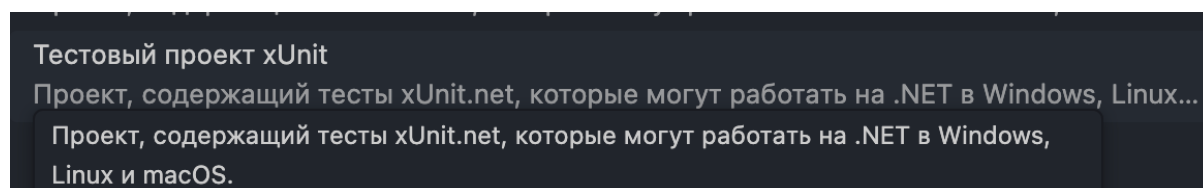
Объяснение на практике: Тестирование простого калькулятора

Возьмем уже созданный и улучшенный нами простой калькулятор. Добавим новый проект в папку с нашим калькулятором. Для этого нажмем пкм на пустое пространство в папке с нашими для наших проектов.

ВАЖНО: Нельзя создавать проект внутри другого проекта, например, не должно быть два файла с расширением `.csproj` в одной папке. (То есть на пустое пространство нажимаете и там создаете новый проект).



Выбираем шаблон проекта



xUnit — это популярный фреймворк с открытым исходным кодом, предназначенный для написания и запуска автоматических модульных (юнит-тестов) в среде .NET для языка C#.

Наш проект с юнит тестом создан и он должен выглядеть внутри вот так:

```
namespace unitTest2;
    0 references
public class UnitTest1
{
    [Fact]
    0 references
    public void Test1()
    {

    }
}
```

Что именно проверяется?

Тестируют логику вычислений, правила валидации, алгоритмы и принятие решений в коде.

Принято делить структуру теста на три шага (AAA — Arrange, Act, Assert):

1. Arrange (Подготовка): Создаем объекты, задаем входные данные.
2. Act (Действие): Вызываем тестируемый метод.
3. Assert (Проверка): Сравниваем результат с тем, который мы ожидали получить.

Главные атрибуты xUnit:

1. [Fact] — обычный тест без параметров для конкретного сценария.
2. [Theory] + [InlineData] — параметризованный тест. Позволяет запустить один и тот же код проверки с разным набором входных данных (например, передать разные суммы и скидки).

Теперь нам нужно связать тест с нашим калькулятором для этого в файле теста с расширением .csproj пишем следующее:

```
<ItemGroup>
  <ProjectReference Include="..\Testirovanie\Testirovanie.csproj" />
</ItemGroup>
```

Тут мы добавляем ссылку на проект с функцией, которую хотим протестировать

В файле, где написан наш тест пишем следующее:

```
1  using Testirovanie;
2
3  namespace unitTest1;
4
5  0 references
6  public class UnitTest1
7  {
8      [Fact]
9      0 references
10     public void Add_Test()
11     {
12         decimal a = 2m;
13         decimal b = 3m;
14
15         decimal result = MainWindow.Add(a, b);
16
17         Assert.Equal(5m, result);
18     }
19 }
```

В файле теста мы прописываем `using Testirovanie`, чтобы использовать функцию `Add` из нашего проекта. Создаем метод `Add_Test` с атрибутом `[Fact]`, где создаем две переменные, хранящие тестовые данные, а переменная `result` хранит сумму двух чисел, которую возвращает функция `Add`. `Assert.Equal(5m, result)` мы сравниваем правильный вариант с вариантом, который вернула функция.

Тест написан, чтобы его запустить в терминале по адресу, где находится исполняемый файл пишем **`dotnet build -> dotnet test`**

```
1 0.85
unitTest1 % dotnet test
```

После этого в терминал выведет следующее:

```
Test summary: total: 1, failed: 0, succeeded: 1, skipped: 0, duration: 0.6s  
Build succeeded with 4 warning(s) in 1.4s
```

Это означает, что наш тест работает корректно. В выводе указывается количество тестов, проваленных, успешных, пропущенных и затраченное время на тестирование.

Справка (Пример работы)

MainWindow.axaml

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
        x:Class="Testirovanie.MainWindow"
        Title="Калькулятор сложения"
        Width="520" Height="200"
        WindowStartupLocation="CenterScreen">

    <StackPanel Orientation="Horizontal"
                HorizontalAlignment="Center"
                VerticalAlignment="Center"
                Spacing="12">

        <TextBox x:Name="FirstNumber"
                 Width="120"
                 FontSize="18"
                 TextAlignment="Center"
                 Watermark="Число"
                 TextChanged="OnNumberTextChanged" />

        <TextBlock Text="+"
                  FontSize="24"
                  VerticalAlignment="Center" />

        <TextBox x:Name="SecondNumber"
                 Width="120"
                 FontSize="18"
                 TextAlignment="Center"
                 Watermark="Число"
                 TextChanged="OnNumberTextChanged" />

        <Button x:Name="CalculateButton"
                Content="="
                FontSize="24"
                VerticalAlignment="Center"
                Click="OnCalculateClick" />

        <TextBlock x:Name="ResultText"
                  FontSize="24"
                  VerticalAlignment="Center"
                  MinWidth="80"
                  TextAlignment="Center" />

    </StackPanel>
</Window>
```

MainWindow.axaml.cs

```
using System.Globalization;
using Avalonia.Controls;
using Avalonia.Interactivity;

namespace Testirovanie;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    public static decimal Add(decimal a, decimal b)
    {
        return a + b;
    }

    private void OnCalculateClick(object? sender, RoutedEventArgs e)
    {
        if (decimal.TryParse(FirstNumber.Text ?? string.Empty, NumberStyles.Float,
            CultureInfo.InvariantCulture, out var a) &&
            decimal.TryParse(SecondNumber.Text ?? string.Empty, NumberStyles.Float,
            CultureInfo.InvariantCulture, out var b))
        {
            ResultText.Text = Add(a, b).ToString(CultureInfo.InvariantCulture);
        }
    }

    private void OnNumberTextChanged(object? sender, TextChangedEventArgs e)
    {
        if (sender is not TextBox textBox) return;

        var filtered = new System.Text.StringBuilder(textBox.Text?.Length ?? 0);
        var hasSeparator = false;

        foreach (var ch in textBox.Text ?? string.Empty)
        {
            if (char.IsDigit(ch))
            {
                filtered.Append(ch);
            }
            else if ((ch == '.' || ch == ',') && !hasSeparator)
            {
                filtered.Append('.');
                hasSeparator = true;
            }
        }

        if (filtered.ToString() != textBox.Text)
        {
            textBox.Text = filtered.ToString();
            textBox.CaretIndex = textBox.Text.Length;
        }
    }
}
```

Практическое задание

1. Провести тестирование, которые описаны в ваших чек-листах
2. Запустить и проверить вывод в терминале после запуска тестов
3. При сдаче работ объяснить, как выполняли и показать результат